Projekt: Browary Bilba – Algorytm Forda-Fulkersona z BFS

Autor: Łukasz Pniewski

Data wykonania testów: 26.05.2025

Środowisko testowe

Język programowania: C++

Platforma: Linux / Windows

Kompilator: g++

Biblioteki: “doctest.h”

Framework testowy: doctest v2

Cel testów

Celem jest weryfikacja poprawności implementacji algorytmu Forda-Fulkersona, mianowicie jego wersji z wyszukiwaniem ścieżek za pomocą algorytmu BFS (Breadth-First Search). Testy mają sprawdzić poprawność wypełnienia tablicy poprzedników za pomocą BFS, wyników znalezienia ścieżki oraz obliczonego przepływu.

Zakres testów

1) bfs(graf, source, target, rodzic)

- poprawne stwierdzenie o znalezieniu ścieżki,

- poprawne stwierdzenie o braku ścieżki

- odpowiednie wypełnianie tablicy ***rodzic***, przetrzymującą znalezioną ścieżkę w postaci przypisania poprzedników każdemu wierzchołkowi

2) fordFulkerson(graf, source, target)

- znalezienie oczekiwanego maksymalnego przepływu przez sieć rezydualną, reprezentowaną jako macierz sąsiedztwa (vector 2D), a podaną jako argument funkcji fordFulkerson

- w razie braku przepływu, zwrócenie 0

Wykonane testy

1) BFS

a) Minimalny przepływ, jedna ścieżka

Testuje poprawność znalezienia jedynej możliwej ścieżki oraz proste wypełnianie tablicy poprzedników.

b) Brak połączenia z browarem

Testuje, czy algorytm zwróci stan o braku ścieżki powiększającej. Przy tym upewnia się, że punkty do których algorytm nie dotarł nie posiadają swoich poprzedników.

c) Brak polaczenia karczmy z target

Testuje sytuację, w której mimo przejścia przez graf, w ostatnim momencie nie ma dojścia do ***target***. Algorytm powinien zwrócić *false* oraz tablica poprzedników powinna mieć ich ustawionych nawet dla wierzchołków, które akurat nie były częścią danej znajdowanej ścieżki.

d) Graf zawierający cykl + różne przepustowości

Ma na celu sprawdzenie, czy algorytm radzi sobie z cyklem i różnymi przepustowościami w podanym grafie. Powinien zwrócić *true* oraz poprawnie zakodowaną tablicę poprzedników.

e) Graf z krawędzią odwrotną

Testuje, czy w razie wystąpienia krawędzi odwrotnej, algorytm dalej szuka poprawnej ścieżki powiększającej.

f) Wszystkie możliwe połączenia w grafie

Czy przy wystąpieniu każdej dozwolonej według treści zadania krawędzi, algorytm poprawnie szuka ścieżki powiększającej.

2) Ford-Fulkerson

a) Przepływ 1, jedna ścieżka

Testuje najprostszy przypadek, gdzie istnieje tylko jedna ścieżka. Przepływ powinien wynosić 1.

b) Graf bardziej złożony

Testuje przypadek grafu z różnymi przepustowościami, gdzie znalezionych będzie wiele ścieżek.

c) Wszystkie możliwe połączenia w grafie

Przypadek, gdzie krawędzie występują w każdym możliwym miejscu (na podstawie treści zadania). Algorytm powinien zwrócić przepływ 2, ze względu na możliwe dwie krawędzie prowadzące do ***target***, z przepustowością 1.

d) Wąskie gardło

Testuje sytuację, w której przepustowość jednej z krawędzi znacząco ogranicza całkowity możliwy przepływ, mimo istnienia wielu ścieżek.

e) Użycie krawędzi odwrotnej w ścieżce

Sprawdza, czy algorytm poprawnie wykorzystuje odwrotne krawędzie w celu zwiększenia przepływu.

f) Graf zawierający cykl

Testuje działanie algorytmu w obecności cyklu w grafie – upewniając się, że nie dochodzi do zapętleń, ani błędów.

g) Brak przepływu

Sprawdza sytuację, w której nie istnieje żadna ścieżka prowadząca od źródła do ujścia – maksymalny przepływ powinien wynosić 0.

h) Graf z wielkimi i małymi przepustowościami

Testuje stabilność algorytmu na grafie zawierającym znaczne różnice w przepustowościach poszczególnych krawędzi. Upewnia się, że przepływ nie będzie większy niż powinien.

i) Wiele zbędnych wierzchołków w grafie

Sprawdza odporność algorytmu na obecność nieużywanych wierzchołków, które nie powinny mieć wpływu na przepływ oraz zwiększają złożoność grafu.

Wyniki testów

Testy przeprowadzone – 15

Testy zakończone sukcesem – 15

Testy zakończone niepowodzeniem – 0

Przeprowadzone asercje – 53

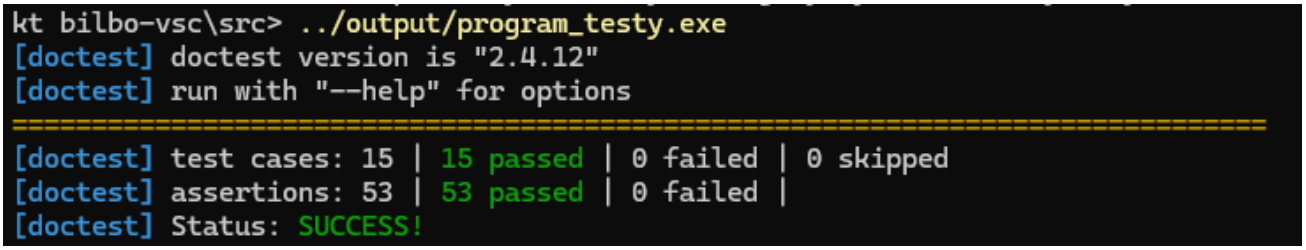
Asercje zakończone sukcesem – 53

Asercje zakończone niepowodzeniem – 0

Wykryte defekty: Brak

Pokrycie funkcjonalne: Wysokie (na podstawie analizy testowanych funkcjonalności)

Wynik z konsoli:



Podsumowanie

Przeprowadzone testy potwierdziły poprawność działania algorytmu Forda-Fulkersona z użyciem BFS. Algorytm prawidłowo znajduje maksymalny przepływ w przypadkach prostych, jak i w tych zaawansowanych. Wszystkie scenariusze testowe zakończyły się sukcesem, a otrzymane wyniki były zgodne z oczekiwaniami. Implementacja działa stabilnie i spełnia założone wymagania funkcjonalne.